
Real-Time Object Detection and Counting from Video Streams

Full Report (Part 2)

P. O. Adepoju

AI/ML Engineering Portfolio Project

Model Family: YOLOv8 (Nano / Small / Medium)
Dataset: COCO 80-class pretrained weights
Task: Multi-class object detection & counting
Date: June 19, 2026

Contents

1	Results	2
1.1	Single-Frame Detection Results	2
1.2	Video-Level Inference Statistics	2
1.3	Object Counting Results	3
1.4	Benchmark Results	4
1.5	Model Comparison and Deployment Recommendations	5
2	Discussion	6
2.1	Limitations	6
2.2	Future Extensions	7
3	Ethical Considerations	9
3.1	Privacy and Surveillance	9
3.2	Dataset Bias and Fairness	9
3.3	Automation and Human Oversight	10
3.4	Dual-Use Risks	11
4	Conclusion	12
	References	14

Chapter 1

Results

1.1 Single-Frame Detection Results

Running YOLOv8-small on the middle frame of the traffic video (frame index 750, confidence threshold 0.25) produced 8 detections in 407.4 ms (first-call, JIT-affected latency). The full detection table is reproduced in Table 3.1 of Part 1. Key findings:

- **High-confidence cars.** The four most prominent cars (confidence 0.78–0.91) are clearly visible mid-frame, well-separated from other objects, and close enough to the camera that they occupy a substantial fraction of the frame. The model assigns confident predictions to these instances.
- **Low-confidence car.** Detection 7 (car, confidence 0.37) is a large vehicle in the centre-left of the frame that the model partially classifies as a car. This vehicle occupies the same bounding box as Detection 6 (truck, 0.38), producing a near-duplicate pair at nearly equal class probabilities. The model is genuinely uncertain between “car” and “truck” for this object.
- **Truck detections at lower confidence.** The two truck detections (0.38, 0.48) have lower confidence than the car detections. This is consistent with the observation that trucks in this video are commercial vans rather than the articulated lorries that dominate the COCO “truck” training distribution.

1.2 Video-Level Inference Statistics

Processing all 1,501 frames of the 60-second traffic video with YOLOv8-small on CPU produced the following pipeline-level statistics:

Table 1.1: Full-video inference statistics (YOLOv8-small, CPU, 1,501 frames, 1920×1080 input). The mean FPS of 4.8 represents a 5.2× real-time lag ($25 \text{ FPS} \div 4.8 \text{ FPS}$) accumulated over the 60-second video.

Metric	Value
Total frames processed	1,501
Total detections	10,725
Mean detections per frame	7.15
Mean inference latency	222.0 ms
Median inference latency	201.2 ms
Std deviation (latency)	120.8 ms
p95 latency	313.7 ms
p99 latency	464.7 ms
Max latency (single frame)	4,108.6 ms
Mean FPS	4.8
Total wall-clock time	$\approx 370 \text{ s}$ ($\approx 6 \text{ min } 10 \text{ s}$)
Output video size	73.9 MB
Detection JSON size	4.5 MB

The maximum single-frame latency of 4,108.6 ms is a 20× spike above the mean. These extreme outliers are attributable to OS-level interruptions during the ≈ 6 -minute processing run: disk I/O, memory paging, or background process scheduling can cause a several-second stall in a CPU-bound PyTorch inference thread.

1.3 Object Counting Results

Table 1.2: Per-class object count summary over all 1,501 frames (60 seconds) of the traffic video, processed with YOLOv8-small, confidence threshold 0.25.

Class	Total	Mean/Frame	Peak	Frames Present	% Frames
car	7,776	5.18	9	1,501	100.0%
truck	2,044	1.36	5	1,241	82.7%
person	745	0.50	5	468	31.2%
motorcycle	155	0.10	2	150	10.0%
bus	5	0.003	1	5	0.3%

Car Class. With 7,776 total appearances and 100% frame presence, the car class dominates the video entirely. A mean of 5.18 cars per frame and a peak of 9 in a single frame indicates a moderately dense urban traffic scene. The counting stability metrics for the car class underscore the model’s consistency:

- Mean frame-to-frame change: $|\Delta \text{count}| = 0.18 \text{ cars/frame}$.
- Maximum frame-to-frame change: 2 cars.

A mean absolute change of 0.18 cars per frame (at 25 FPS, approximately 4.5 cars per second) indicates highly stable detections, the model consistently detects the same vehicles in adjacent frames.

Truck Class. Trucks appear in 82.7% of frames (1,241 of 1,501), suggesting sustained but not universal presence throughout the video. The 17.3% of frames without truck detections likely correspond to segments where the camera angle shows only the near lanes, where trucks are absent.

Person Class. Persons appear in 31.2% of frames. The temporal pattern shows person detections concentrated in specific time windows, consistent with pedestrian activity near a crossing or footpath visible at certain points in the road scene.

Motorcycle and Bus. Motorcycles (10.0% frames, max 2 in a single frame) and buses (0.3% frames, max 1) are rare. The motorcycle appearances cluster in a specific time window (approximately frames 700–850), consistent with a segment of road where two-wheeled vehicles are present. The single bus appearance (5 frames) is a marginal detection at very low confidence, which would be filtered at a threshold of $\tau \geq 0.3$.

1.4 Benchmark Results

Table 1.3: Complete benchmark comparison across three YOLOv8 variants (CPU, 640×480 px input, 100 benchmark frames, 10 warm-up frames discarded). All latency values in ms; memory in MB.

Model	FPS	Mean (ms)	Med. (ms)	p95 (ms)	p99 (ms)	Std (ms)	Mem Δ (MB)	Size (MB)
YOLOv8-nano	9.58	104.4	107.0	124.3	145.7	13.8	1.26	6.2
YOLOv8-small	4.37	229.1	216.3	276.0	295.2	24.6	4.98	21.5
YOLOv8-medium	1.20	831.7	653.3	1388.5	4119.1	625.7	0.48	49.7

FPS Scaling. FPS decreases monotonically with model scale: nano (9.58) $\rightarrow$ small (4.37) $\rightarrow$ medium (1.20). The scaling ratios are:

- Nano to small: $9.58/4.37 = 2.19\times$ FPS reduction for a $3.53\times$ parameter increase.
- Small to medium: $4.37/1.20 = 3.64\times$ FPS reduction for a $2.32\times$ parameter increase.

The super-linear FPS degradation from small to medium ($3.64\times$ FPS drop for only $2.32\times$ more parameters) is attributable to the medium model’s weight tensors exceeding the CPU’s L3 cache capacity, causing frequent off-chip DRAM accesses. This effect is discussed in Section ??.

Latency Distribution Shape. The three models exhibit qualitatively different latency distributions:

- **YOLOv8-nano:** Tight, near-symmetric distribution (std = 13.8 ms, $\approx 13\%$ of mean). The p99

(145.7 ms) is only 40% above the mean (104.4 ms), indicating very predictable behaviour. This makes nano the only variant among the three that could safely satisfy a 150 ms per-frame latency SLO on this hardware.

- **YOLOv8-small:** Mildly right-skewed (std = 24.6 ms, $\approx 11\%$ of mean). Mean (229.1 ms) and median (216.3 ms) are close, suggesting only modest outlier influence. The p99 (295.2 ms) is 29% above the mean.
- **YOLOv8-medium:** Dramatically right-skewed (std = 625.7 ms, $\approx 75\%$ of mean). The mean (831.7 ms) exceeds the median (653.3 ms) by 27%. The p99 (4,119 ms) is nearly 5 $\times$ the mean, indicating that ≈ 1 in 100 frames takes over 4 seconds. In a 25 FPS video stream, this would produce a backlog of $25 \times 4 = 100$ unprocessed frames from a single stall event.

1.5 Model Comparison and Deployment Recommendations

Table 1.4: Final model comparison with deployment scenario recommendations.

Model	Params (M)	FPS	Mean (ms)	p95 (ms)	Mem Δ (MB)	Deployment
YOLOv8-nano	3.16	9.58	104.4	124.3	1.26	Near real-time CPU; edge device
YOLOv8-small	11.17	4.37	229.1	276.0	4.98	Batch/offline; low-latency-tolerant
YOLOv8-medium	25.90	1.20	831.7	1388.5	0.48	GPU only; batch processing

No variant achieves the 25 FPS native video frame rate on CPU. YOLOv8-nano at 9.58 FPS comes closest, operating at a 2.6 $\times$ real-time lag. For applications tolerant of sub-real-time throughput (surveillance dashboards updating at 5–10 FPS, counting systems that process 1 frame per second), nano on CPU is entirely practical. GPU deployment would push all three models well above 25 FPS:

Table 1.5: Estimated GPU FPS from Ultralytics published benchmarks on NVIDIA T4 GPU (for reference; not measured in this project).

Model	CPU FPS (this project)	T4 GPU FPS (est.)
YOLOv8-nano	9.58	$\approx 250\text{--}400$
YOLOv8-small	4.37	$\approx 100\text{--}180$
YOLOv8-medium	1.20	$\approx 50\text{--}80$

These GPU estimates illustrate that the choice of model scale becomes less critical on GPU: even the medium model would likely surpass 25 FPS, and the nano model would enable multi-stream processing.

Chapter 2

Discussion

2.1 Limitations

Limitation 1: No Temporal Object Tracking. The most significant limitation is that this system counts *frame-level appearances*, not unique objects. A car that drives through the entire 60-second video contributes approximately $5.18 \times 1,501 \approx 7,776$ to the total count, not 1. For traffic flow measurement (vehicles per minute through a junction), unique-object counting via a temporal tracker is required. SORT (Bewley, Ge, Ott, Ramos, & Upcroft, 2016) and ByteTrack (Zhang et al., 2022) are both well-suited to this task and integrate naturally with the existing `inference.py` output format.

Limitation 2: COCO Class Coverage. The 80 COCO classes cover everyday objects from a diverse but predominantly Western urban dataset. Objects outside these 80 classes are invisible to the pretrained model. Any deployment in a domain with custom objects requires collecting a labelled dataset and fine-tuning.¹

Limitation 3: CPU-Only Benchmarks. All benchmark results are specific to the test hardware (CPU model, clock speed, cache size, RAM speed, thermal conditions). The absolute FPS values are not comparable across machines. GPU benchmarks (using the same controlled protocol) would complete the performance picture and are left for future work (Section 2.2).

Limitation 4: No mAP Evaluation. The detection count proxy used in *Notebook 07* is not a substitute for COCO mAP evaluation on val2017. The proxy cannot distinguish between increased true detections and increased false positives, making it an incomplete measure of model quality. Formal mAP evaluation on COCO val2017 would require downloading the 1 GB validation set and ground-truth JSON annotations.²

Limitation 5: Single Video Source. Results on one 60-second daylight urban traffic video do not generalise. Detection rates, class distributions, and counting stability would differ substantially in night-time footage, indoor scenes, high-density crowds, or rural roads. A robust evaluation would test across 5–10 videos covering different lighting conditions, vehicle types, and camera perspectives.

Limitation 6: Global Confidence Threshold. The single global threshold ($\tau = 0.25$) applies equally to all 80 COCO classes. Class-specific thresholds (e.g., a lower threshold for persons to maximise recall in

¹LabelImg is a widely used open-source annotation tool: <https://github.com/heartexlabs/labelImg>. Roboflow provides a managed annotation and dataset management platform: <https://roboflow.com/>.

²COCO evaluation toolkit: <https://github.com/cocodataset/cocoapi>.

safety applications, a higher threshold for buses to reduce false alarms) could improve the precision–recall balance for specific use cases without retraining.

2.2 Future Extensions

1. Temporal Tracking (SORT / ByteTrack). Integrate a multi-object tracker to assign persistent identities across frames, enabling unique-vehicle counts and track-based analytics (speed estimation, trajectory analysis, virtual line crossing). ByteTrack (Zhang et al., 2022) is the state-of-the-art tracker that pairs naturally with YOLO detections through a Kalman-filter prediction and Hungarian-algorithm assignment pipeline.³

2. GPU Benchmarking and ONNX/TensorRT Export. Complete the speed comparison with GPU benchmarks (CUDA, NVIDIA T4 / RTX series), including ONNX Runtime and TensorRT inference modes. Export the model:

```
model.export(format="engine", device=0) # TensorRT FP16
```

TensorRT achieves 3–10× speedup over PyTorch inference on compatible NVIDIA GPUs.

3. Domain Fine-Tuning. Annotate 500 images from the target domain (e.g., the specific traffic camera angle) with LabelImg or Roboflow, construct a custom.yaml dataset file, and fine-tune YOLOv8-small for 100 epochs. Evaluate with COCO mAP metrics on a held-out validation set. Expected outcome: 5–20% mAP improvement over zero-shot on the target domain.

4. INT8 Quantisation. Apply post-training INT8 quantisation to reduce model size by $\approx 4\times$ and inference latency by $\approx 2\times$ at a typical 3–8% mAP cost. This is particularly valuable for edge deployment where both memory and compute are constrained.

5. Virtual Line Crossing and ROI Counting. Implement zone-based analytics: define virtual lines in the camera’s field of view and count objects that cross each line per unit time (vehicles/minute). This requires centroid tracking (from the temporal tracker) and line-intersection logic:

```
def crossed_line(prev_centroid, curr_centroid, line_start, line_end):  
    return segments_intersect(  
        prev_centroid, curr_centroid, line_start, line_end  
    )
```

Virtual line crossing is the standard metric for junction throughput measurement in traffic engineering.⁴

³ByteTrack GitHub repository: <https://github.com/ifzhang/ByteTrack>. A practical tutorial on SORT with YOLOv8 is available at <https://learnopencv.com/object-tracking-with-yolov8-and-bytetrack/>.

⁴OpenCV’s virtual line crossing tutorial: https://docs.opencv.org/4.x/d9/d61/tutorial_py_morphological_ops.html.

6. Model Cascading. Run nano first; trigger medium only on frames where all detections have confidence < 0.6 . This adaptive strategy achieves the speed of nano for clear frames while applying the accuracy of medium for ambiguous ones. Expected result: mean FPS close to nano with detection quality approaching medium on hard frames.

Chapter 3

Ethical Considerations

3.1 Privacy and Surveillance

The person class in COCO detects human presence as a bounding box. In a traffic monitoring context, this is benign: knowing that 0.50 persons per frame are visible on a road segment does not identify anyone. However, the same pipeline, pointed at a building entrance, a park, or a residential street, becomes a tool for tracking the movements of identifiable individuals.

Applicable Legal Frameworks. Operators deploying video object detection systems are subject to:

- **GDPR (EU, 2018):** Article 4 defines video footage of an identifiable person as personal data.¹ Article 6 requires a lawful basis for processing. Article 35 requires a Data Protection Impact Assessment (DPIA) for high-risk surveillance processing. Violations carry fines of up to 4% of global annual turnover.
- **NDPR (Nigeria, 2019):** The Nigeria Data Protection Regulation mirrors GDPR principles, requiring notice, consent, and data minimisation for personal data processing.²
- **CCPA (California, USA, 2020):** Grants residents the right to know what personal data is collected and to opt out of its sale.

The system built in this project processes a public-domain video with no identifiable individuals visible at sufficient resolution. However, any operator adapting the pipeline for a real deployment must conduct a legal review under the applicable jurisdiction’s data protection framework before going live.

3.2 Dataset Bias and Fairness

The COCO dataset (Lin et al., 2014) was assembled primarily from Flickr images, which over-represent certain geographic regions (North America, Western Europe), object appearances (cars typical of those markets), and environmental conditions (daylight, good weather). Several documented biases are relevant:

- **Vehicle type bias.** The COCO “truck” class was annotated primarily on articulated lorries (18-wheelers). Commercial vans, minibuses, and pickup trucks common in non-Western contexts may produce lower-confidence detections or be misclassified as cars.

¹GDPR Article 4 on personal data: <https://gdpr.eu/article-4-definitions/>.

²Nigeria Data Protection Regulation: <https://ndpb.gov.ng/Files/NigeriaDataProtectionRegulation.pdf>.

- **Pedestrian appearance bias.** Detection performance for the “person” class may differ across clothing types, body sizes, and skin tones due to biases in the Flickr training distribution. This is particularly important for safety-critical applications (pedestrian detection at crossings) where equal performance across demographic groups is an ethical requirement.³
- **Geographic and lighting bias.** Night-time imagery, dust, haze, and non-Western infrastructure (road markings, traffic signs) are under-represented in COCO. Detection rates may be systematically lower in these conditions.

Mitigation. Domain-specific fine-tuning on a representative annotated dataset is the most effective mitigation for bias in a specific deployment context. Demographic fairness evaluation (measuring detection recall separately by demographic group) should be performed before deploying any people-detection system in a context where outcomes affect individuals.

3.3 Automation and Human Oversight

Counting systems used in high-stakes decisions should not operate as fully autonomous decision-makers.

Pedagogical Note: The Automation Bias Risk

Automation bias is the tendency of human operators to over-trust automated outputs, especially when those outputs are presented with apparent precision (e.g., “17 persons detected”) rather than uncertainty (“between 14 and 20 persons detected, at 80% confidence”).^a A count that is accurate on average but has a standard deviation of ± 1.5 objects per frame is not a reliable basis for a hard threshold trigger (e.g., “if count > 10, lock the door”) without human review. Any threshold-triggered action downstream of an automated detection system requires human confirmation before enforcement.

^aAutomation bias in human-machine systems: Skitka, Mosier, and Burdick (1999).

Recommended Oversight Mechanisms.

1. Confidence intervals or standard deviations should accompany every reported count.
2. Threshold-triggered alerts should include a human review step before any physical action (locking doors, triggering alarms, initiating evacuation).
3. Audit logs of all detections and counts should be retained and periodically reviewed by responsible personnel.
4. System accuracy should be re-evaluated periodically as environmental conditions change (new vehicles, new infrastructure, seasonal lighting).

³A comprehensive study of demographic disparities in pedestrian detection systems: Wilson, Hoffman, and Morgenstern (2019). See also the AI Fairness 360 toolkit: <https://aif360.mybluemix.net/>.

3.4 Dual-Use Risks

The pipeline developed in this project can serve legitimate purposes (traffic analytics, retail occupancy) or harmful ones (mass surveillance, targeted tracking of specific individuals or groups). This project **explicitly excludes**:

- Facial recognition or biometric identification of any kind.
- Ethnic, racial, or demographic profiling.
- Tracking of named individuals across locations or time.
- Any use case that bypasses individual consent in jurisdictions requiring it.

Operators deploying the system are solely responsible for ensuring their use case is lawful, ethical, and consistent with the rights of the people in the footage. The author and contributors to this codebase accept no responsibility for harmful downstream use. Researchers and practitioners are encouraged to review the Partnership on AI's guidelines on facial recognition and surveillance before deployment.⁴

⁴Partnership on AI responsible AI guidelines: <https://partnershiponai.org/resources/>.

Chapter 4

Conclusion

This report has presented the design, implementation, evaluation, and contextualisation of a complete real-time object detection and counting system for video streams. The system was built around YOLOv8 (Jocher, Chaurasia, & Qiu, 2023) pretrained on the COCO 80-class dataset (Lin et al., 2014), implemented as a nine-notebook reproducible pipeline backed by a modular Python package (`detection_system`), and evaluated on a 60-second urban traffic video.

Engineering Achievements. The pipeline successfully processes all 1,501 frames of a 1920×1080 video, producing an annotated output video (73.9 MB), a structured detection record (4.5 MB JSON), per-class count time-series CSVs, publication-quality figures (300 DPI PNG and vector PDF), and a deployable Streamlit interactive dashboard. The codebase is fully modular: all inference, counting, benchmarking, and visualisation logic lives in the `detection_system` package, tested independently of the notebooks and reusable in any downstream application.

Scientific Findings. The benchmarking experiment produced three clear operating points on the speed–accuracy–memory curve under controlled CPU conditions:

- **YOLOv8-nano** (3.16M parameters, 6.2 MB): 9.58 FPS at 104.4 ms mean latency, with a tight, predictable distribution (p99 = 145.7 ms). The recommended choice for any CPU deployment requiring near-real-time throughput.
- **YOLOv8-small** (11.17M parameters, 21.5 MB): 4.37 FPS at 229.1 ms. Practical for batch processing, low-update-rate monitoring, and applications tolerant of a ~5-second real-time lag.
- **YOLOv8-medium** (25.90M parameters, 49.7 MB): 1.20 FPS at 831.7 ms with a heavy-tailed distribution (p99 = 4,119 ms). Unsuitable for any real-time CPU application; practical only on GPU or in non-time-sensitive batch workflows.

The FPS degradation is super-linear relative to parameter count (small→medium: 3.64× FPS drop for 2.32× parameter increase), explained by the medium model’s weight tensors exceeding CPU L3 cache capacity and triggering repeated off-chip DRAM accesses.

Object counting on the 60-second traffic video revealed 10,725 total detections: 7,776 car appearances (100% frame presence), 2,044 truck (82.7%), 745 person (31.2%), 155 motorcycle (10.0%), and 5 bus (0.3%). Frame-level counting stability was high: the car class showed a mean absolute frame-to-frame change of only 0.18 objects/frame, indicating consistent cross-frame detection despite the absence of

temporal tracking.

Limitations and Path Forward. Three limitations warrant emphasis: (1) the absence of temporal tracking means total counts represent frame-level appearances rather than unique objects; (2) no GPU benchmarks were conducted; and (3) formal mAP evaluation on COCO val2017 was not performed. These are well-defined gaps that constitute the immediate next steps for future work, as described in Section 2.2.

Broader Significance. This project demonstrates that a production-quality, deployable computer vision pipeline can be built using exclusively open-source tools—YOLOv8, OpenCV, PyTorch, Ultralytics, Streamlit, and Python’s scientific ecosystem—within a single week. The full pipeline from raw video to annotated output, count statistics, benchmark tables, and interactive dashboard requires fewer than 2,000 lines of code (including the Streamlit app).

More importantly, the systematic benchmarking methodology—warm-up protocol, controlled input conditions, tail-latency reporting, explicit discussion of measurement artefacts (memory pool effects, JIT warm-up)—provides a template that students and practitioners can apply to any ML inference system, not just object detection. Reporting the mean without the distribution, or reporting the first-call latency without warm-up, are common mistakes in ML engineering benchmarks; this project explicitly avoids both.

The study of real-time inference is not complete with the work presented here. GPU deployment, temporal tracking, domain fine-tuning, model quantisation, and multi-model cascading all remain as productive extensions. Each of these extensions is tractable from the foundation established in this project, and each would deepen the practitioner’s understanding of the trade-offs that govern the deployment of ML systems in the real world.

“In God we trust; all others must bring data.”

— W. Edwards Deming

References

- Bewley, A., Ge, Z., Ott, L., Ramos, F., & Upcroft, B. (2016). Simple online and realtime tracking. *Proceedings of the IEEE International Conference on Image Processing (ICIP)*, 3464–3468. Retrieved from <https://arxiv.org/abs/1602.00763> doi: 10.1109/ICIP.2016.7533003
- Jocher, G., Chaurasia, A., & Qiu, J. (2023). *Ultralytics YOLOv8*. <https://github.com/ultralytics/ultralytics>. (Version 8.0.0. MIT Licence. Model documentation at <https://docs.ultralytics.com/>)
- Lin, T.-Y., Maire, M., Belongie, S., Hays, J., Perona, P., Ramanan, D., ... Zitnick, C. L. (2014). Microsoft COCO: Common objects in context. In *Proceedings of the european conference on computer vision (eccv)* (pp. 740–755). Retrieved from <https://arxiv.org/abs/1405.0312> doi: 10.1007/978-3-319-10602-1_48
- Skitka, L. J., Mosier, K. L., & Burdick, M. (1999). Does automation bias decision-making? *International Journal of Human-Computer Studies*, 51(5), 991–1006. doi: 10.1006/ijhc.1999.0252
- Wilson, B., Hoffman, J., & Morgenstern, J. (2019). Predictive inequity in object detection. *arXiv preprint arXiv:1902.11097*. Retrieved from <https://arxiv.org/abs/1902.11097>
- Zhang, Y., Sun, P., Jiang, Y., Yu, D., Weng, F., Yuan, Z., ... Wang, X. (2022). ByteTrack: Multi-object tracking by associating every detection box. In *Proceedings of the european conference on computer vision (eccv)* (pp. 1–21). Retrieved from <https://arxiv.org/abs/2110.06864> doi: 10.1007/978-3-031-20177-8_1